```
    b=(x-xx[klo])/h;                    Cubic spline polynomial is now evaluated.
    y=a*yy[klo]+b*yy[khi]+((a*a*a-a)*y2[klo]
        +(b*b*b-b)*y2[khi])*(h*h)/6.0;
    return y;
}
```

Typical use looks like this:

```
Int n=...;
VecDoub xx(n), yy(n);
...
Spline_interp myfunc(xx,yy);
```

and then, as often as you like,

```
Doub x,y;
...
y = myfunc.interp(x);
```

Note that no error estimate is available.

CITED REFERENCES AND FURTHER READING:

De Boor, C. 1978, *A Practical Guide to Splines* (New York: Springer).

Ueberhuber, C.W. 1997, *Numerical Computation: Methods, Software, and Analysis*, vol. 1 (Berlin: Springer), Chapter 9.

Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall), §4.4 – §4.5.

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §2.4.

Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), §3.8.

3.4 Rational Function Interpolation and Extrapolation

Some functions are not well approximated by polynomials but *are* well approximated by rational functions, that is quotients of polynomials. We denote by $R_{i(i+1)\ldots(i+m)}$ a rational function passing through the $m+1$ points $(x_i, y_i), \ldots, (x_{i+m}, y_{i+m})$. More explicitly, suppose

$$R_{i(i+1)\ldots(i+m)} = \frac{P_\mu(x)}{Q_\nu(x)} = \frac{p_0 + p_1 x + \cdots + p_\mu x^\mu}{q_0 + q_1 x + \cdots + q_\nu x^\nu} \tag{3.4.1}$$

Since there are $\mu + \nu + 1$ unknown p's and q's (q_0 being arbitrary), we must have

$$m + 1 = \mu + \nu + 1 \tag{3.4.2}$$

In specifying a rational function interpolating function, you must give the desired order of both the numerator and the denominator.

Rational functions are sometimes superior to polynomials, roughly speaking, because of their ability to model functions with poles, that is, zeros of the denominator of equation (3.4.1). These poles might occur for real values of x, if the function

to be interpolated itself has poles. More often, the function $f(x)$ is finite for all finite *real* x but has an analytic continuation with poles in the complex x-plane. Such poles can themselves ruin a polynomial approximation, even one restricted to real values of x, just as they can ruin the convergence of an infinite power series in x. If you draw a circle in the complex plane around your m tabulated points, then you should not expect polynomial interpolation to be good unless the nearest pole is rather far outside the circle. A rational function approximation, by contrast, will stay "good" as long as it has enough powers of x in its denominator to account for (cancel) any nearby poles.

For the interpolation problem, a rational function is constructed so as to go through a chosen set of tabulated functional values. However, we should also mention in passing that rational function approximations can be used in analytic work. One sometimes constructs a rational function approximation by the criterion that the rational function of equation (3.4.1) itself have a power series expansion that agrees with the first $m+1$ terms of the power series expansion of the desired function $f(x)$. This is called *Padé approximation* and is discussed in §5.12.

Bulirsch and Stoer found an algorithm of the Neville type that performs rational function extrapolation on tabulated data. A tableau like that of equation (3.2.2) is constructed column by column, leading to a result and an error estimate. The Bulirsch-Stoer algorithm produces the so-called *diagonal* rational function, with the degrees of the numerator and denominator equal (if m is even) or with the degree of the denominator larger by one (if m is odd; cf. equation 3.4.2 above). For the derivation of the algorithm, refer to [1]. The algorithm is summarized by a recurrence relation exactly analogous to equation (3.2.3) for polynomial approximation:

$$R_{i(i+1)\ldots(i+m)} = R_{(i+1)\ldots(i+m)} + \frac{R_{(i+1)\ldots(i+m)} - R_{i\ldots(i+m-1)}}{\left(\frac{x-x_i}{x-x_{i+m}}\right)\left(1 - \frac{R_{(i+1)\ldots(i+m)} - R_{i\ldots(i+m-1)}}{R_{(i+1)\ldots(i+m)} - R_{(i+1)\ldots(i+m-1)}}\right) - 1} \tag{3.4.3}$$

This recurrence generates the rational functions through $m+1$ points from the ones through m and (the term $R_{(i+1)\ldots(i+m-1)}$ in equation 3.4.3) $m-1$ points. It is started with

$$R_i = y_i \tag{3.4.4}$$

and with

$$R \equiv [R_{i(i+1)\ldots(i+m)} \quad \text{with} \quad m = -1] = 0 \tag{3.4.5}$$

Now, exactly as in equations (3.2.4) and (3.2.5) above, we can convert the recurrence (3.4.3) to one involving only the small differences

$$\begin{aligned} C_{m,i} &\equiv R_{i\ldots(i+m)} - R_{i\ldots(i+m-1)} \\ D_{m,i} &\equiv R_{i\ldots(i+m)} - R_{(i+1)\ldots(i+m)} \end{aligned} \tag{3.4.6}$$

Note that these satisfy the relation

$$C_{m+1,i} - D_{m+1,i} = C_{m,i+1} - D_{m,i} \tag{3.4.7}$$

which is useful in proving the recurrences

$$
\begin{aligned}
D_{m+1,i} &= \frac{C_{m,i+1}(C_{m,i+1}-D_{m,i})}{\left(\frac{x-x_i}{x-x_{i+m+1}}\right)D_{m,i}-C_{m,i+1}} \\
C_{m+1,i} &= \frac{\left(\frac{x-x_i}{x-x_{i+m+1}}\right)D_{m,i}(C_{m,i+1}-D_{m,i})}{\left(\frac{x-x_i}{x-x_{i+m+1}}\right)D_{m,i}-C_{m,i+1}}
\end{aligned}
\tag{3.4.8}
$$

The class for rational function interpolation is identical to that for polynomial interpolation in every way, except, of course, for the different method implemented in `rawinterp`. See the end of §3.2 for usage. Plausible values for M are in the range 4 to 7.

interp_1d.h

```
struct Rat_interp : Base_interp
```

Diagonal rational function interpolation object. Construct with **x** and **y** vectors, and the number `m` of points to be used locally, then call `interp` for interpolated values.

```
{
    Doub dy;
    Rat_interp(VecDoub_I &xv, VecDoub_I &yv, Int m)
        : Base_interp(xv,&yv[0],m), dy(0.) {}
    Doub rawinterp(Int jl, Doub x);
};

Doub Rat_interp::rawinterp(Int jl, Doub x)
```

Given a value `x`, and using pointers to data `xx` and `yy`, this routine returns an interpolated value `y`, and stores an error estimate `dy`. The returned value is obtained by `mm`-point diagonal rational function interpolation on the subrange `xx[jl..jl+mm-1]`.

```
{
    const Doub TINY=1.0e-99;          A small number.
    Int m,i,ns=0;
    Doub y,w,t,hh,h,dd;
    const Doub *xa = &xx[jl], *ya = &yy[jl];
    VecDoub c(mm),d(mm);
    hh=abs(x-xa[0]);
    for (i=0;i<mm;i++) {
        h=abs(x-xa[i]);
        if (h == 0.0) {
            dy=0.0;
            return ya[i];
        } else if (h < hh) {
            ns=i;
            hh=h;
        }
        c[i]=ya[i];
        d[i]=ya[i]+TINY;          The TINY part is needed to prevent a rare zero-over-zero
    }                                 condition.
    y=ya[ns--];
    for (m=1;m<mm;m++) {
        for (i=0;i<mm-m;i++) {
            w=c[i+1]-d[i];
            h=xa[i+m]-x;          h will never be zero, since this was tested in the initial-
            t=(xa[i]-x)*d[i]/h;       izing loop.
            dd=t-c[i+1];
            if (dd == 0.0) throw("Error in routine ratint");
            This error condition indicates that the interpolating function has a pole at the
            requested value of x.
            dd=w/dd;
            d[i]=c[i+1]*dd;
            c[i]=t*dd;
```

```
        }
        y += (dy=(2*(ns+1) < (mm-m) ? c[ns+1] : d[ns--]));
    }
    return y;
}
```

3.4.1 Barycentric Rational Interpolation

Suppose one tries to use the above algorithm to construct a global approximation on the entire table of values using all the given nodes $x_0, x_1, \ldots, x_{N-1}$. One potential drawback is that the approximation can have poles inside the interpolation interval where the denominator in (3.4.1) vanishes, even if the original function has no poles there. An even greater (related) hazard is that we have allowed the order of the approximation to grow to $N-1$, probably much too large.

An alternative algorithm can be derived [2] that has no poles anywhere on the real axis, and that allows the actual order of the approximation to be specified to be any integer $d < N$. The trick is to make the degree of both the numerator and the denominator in equation (3.4.1) be $N-1$. This requires that the p's and the q's not be independent, so that equation (3.4.2) no longer holds.

The algorithm utilizes the *barycentric form* of the rational interpolant

$$R(x) = \frac{\displaystyle\sum_{i=0}^{N-1} \frac{w_i}{x - x_i} y_i}{\displaystyle\sum_{i=0}^{N-1} \frac{w_i}{x - x_i}} \tag{3.4.9}$$

One can show that by a suitable choice of the weights w_i, *every* rational interpolant can be written in this form, and that, as a special case, so can polynomial interpolants [3]. It turns out that this form has many nice numerical properties. Barycentric rational interpolation competes very favorably with splines: its error is often smaller, and the resulting approximation is infinitely smooth (unlike splines).

Suppose we want our rational interpolant to have approximation order d, i.e., if the spacing of the points is $O(h)$, the error is $O(h^{d+1})$ as $h \to 0$. Then the formula for the weights is

$$w_k = \sum_{\substack{i=k-d \\ 0 \le i < N-d}}^{k} (-1)^k \prod_{\substack{j=i \\ j \ne k}}^{i+d} \frac{1}{x_k - x_j} \tag{3.4.10}$$

For example,

$$\begin{aligned} w_k &= (-1)^k, && d = 0 \\ w_k &= (-1)^{k-1} \left[\frac{1}{x_k - x_{k-1}} + \frac{1}{x_{k+1} - x_k} \right], && d = 1 \end{aligned} \tag{3.4.11}$$

In the last equation, you omit the terms in w_0 and w_{N-1} that refer to out-of-range values of x_k.

Here is a routine that implements barycentric rational interpolation. Given a set of N nodes and a desired order d, with $d < N$, it first computes the weights w_k. Then subsequent calls to `interp` evaluate the interpolant using equation (3.4.9). Note that the parameter `jl` of `rawinterp` is not used, since the algorithm is designed to construct an approximation on the entire interval at once.

The workload to construct the weights is of order $O(Nd)$ operations. For small d, this is not too different from splines. Note, however, that the workload for each subsequent interpolated value is $O(N)$, not $O(d)$ as for splines.

interp_1d.h

```
struct BaryRat_interp : Base_interp
```

Barycentric rational interpolation object. After constructing the object, call `interp` for interpolated values. Note that no error estimate `dy` is calculated.

```
{
    VecDoub w;
    Int d;
    BaryRat_interp(VecDoub_I &xv, VecDoub_I &yv, Int dd);
    Doub rawinterp(Int jl, Doub x);
    Doub interp(Doub x);
};

BaryRat_interp::BaryRat_interp(VecDoub_I &xv, VecDoub_I &yv, Int dd)
        : Base_interp(xv,&yv[0],xv.size()), w(n), d(dd)
```

Constructor arguments are $\mathbf{x}$ and $\mathbf{y}$ vectors of length `n`, and order d of desired approximation.

```
{
    if (n<=d) throw("d too large for number of points in BaryRat_interp");
    for (Int k=0;k<n;k++) {
```

Compute weights from equation (3.4.10).

```
        Int imin=MAX(k-d,0);
        Int imax = k >= n-d ? n-d-1 : k;
        Doub temp = imin & 1 ? -1.0 : 1.0;
        Doub sum=0.0;
        for (Int i=imin;i<=imax;i++) {
            Int jmax=MIN(i+d,n-1);
            Doub term=1.0;
            for (Int j=i;j<=jmax;j++) {
                if (j==k) continue;
                term *= (xx[k]-xx[j]);
            }
            term=temp/term;
            temp=-temp;
            sum += term;
        }
        w[k]=sum;
    }
}
Doub BaryRat_interp::rawinterp(Int jl, Doub x)
```

Use equation (3.4.9) to compute the barycentric rational interpolant. Note that `jl` is not used since the approximation is global; it is included only for compatibility with `Base_interp`.

```
{
    Doub num=0,den=0;
    for (Int i=0;i<n;i++) {
        Doub h=x-xx[i];
        if (h == 0.0) {
            return yy[i];
        } else {
            Doub temp=w[i]/h;
            num += temp*yy[i];
            den += temp;
        }
    }
    return num/den;
}
Doub BaryRat_interp::interp(Doub x) {
```

No need to invoke `hunt` or `locate` since the interpolation is global, so override `interp` to simply call `rawinterp` directly with a dummy value of `jl`.

```
    return rawinterp(1,x);
}
```

It is wise to start with small values of d before trying larger values.